

USTHB — UNIVERSITÉ DES SCIENCES ET DE LA TECHNOLOGIE HOUARI BOUMÉDIÈNE

Faculté d'Électronique et d'Informatique — Département Informatique

Spécialité : Cybersécurité — Niveau : Ing3 (Ingénierie Master)

Crypto Educational Suite

Implémentation Interactive et Pédagogique des Algorithmes de
Chiffrement

RAPPORT DE PROJET ACADÉMIQUE

Module : Cryptographie Appliquée

RÉALISÉ PAR :

BENSLIMANE Salima Lamia

YOUNSI Anfel

KAMIRI Lilia

ANNÉE UNIVERSITAIRE :

2025 – 2026

Table des Matières

Résumé / Abstract	3
1. Introduction	4
2. Architecture Générale du Système	5
3. TP1 : Algorithmes de Chiffrement Classique	7
3.1 Caesar Cipher & Analyse par Indice de Coïncidence	7
3.2 Vigenère Cipher & Test de Kasiski	8
3.3 Affine Cipher & Inversion Modulaire	9
3.4 Hill Cipher & Attaque à Clair Connu	10
3.5 Playfair Cipher & One-Time Pad (Crib Dragging)	11
4. TP2 : Cryptographie Symétrique Moderne	13
4.1 Chiffrement par Flux : RC4 & Faiblesses WEP	13
4.2 Data Encryption Standard (DES / 3DES)	14
4.3 Advanced Encryption Standard (AES)	16
4.4 Algorithmes Avancés : Twofish & Serpent	18
4.5 Algorithmes de Chiffrement RC6 & MARS	20
5. TP3 : Cryptographie Asymétrique & Protocoles	22
5.1 RSA, Chiffrement Hybride & Padding OAEP	22
5.2 Échange de Clés Diffie-Hellman & Attaque MITM	24
5.3 Cryptographie sur Courbes Elliptiques (ECC)	25
5.4 Chiffrement d'ElGamal & Malléabilité	27
6. TP4 : Fonctions de Hachage & Effet Avalanche	28
7. TP5 : Signatures Numériques & Non-Répudiation	31
8. Spécificités Techniques & Moteur de Logging	34
9. Manuel d'Utilisation & Guide d'Installation	36
10. Résultats Expérimentaux & Validation	38
11. Analyse Critique & Comparaison	40
12. Conclusion & Perspectives	42
Références Bibliographiques	43
Annexes	44

Résumé / Abstract

Résumé — Ce rapport présente la conception et la réalisation de la Crypto Educational Suite, une plateforme logicielle interactive et modulaire dédiée à l'enseignement pratique des concepts de la cryptographie classique et moderne. Développée en Python avec l'environnement graphique CustomTkinter, l'application intègre 21 algorithmes cryptographiques répartis de manière rigoureuse en cinq axes majeurs : les chiffrements historiques, les primitives symétriques par blocs et par flux, les cryptosystèmes asymétriques, les fonctions de hachage et les protocoles de signature numérique. L'originalité de ce projet réside dans sa capacité à exposer en temps réel le fonctionnement interne des algorithmes à travers un moteur de logging visuel et pas-à-pas, décomposant chaque opération primitive (XOR, S-Boxes, permutations, planifications de clés). De plus, l'application modélise et exécute les attaques classiques associées à ces algorithmes (cryptanalyse par indice de coïncidence, test de Kasiski, attaque à clair connu sur le chiffrement de Hill, attaques par falsification de signatures et attaques MITM), offrant un outil pédagogique complet pour l'assimilation concrète de la confusion, de la diffusion et de la robustesse mathématique des algorithmes.

Keywords — Cryptographie, Éducation, CustomTkinter, Asymétrique, Symétrique, Attaque, Effet Avalanche, S-Box, USTHB.

1. Introduction

L'enseignement de la cryptographie au sein des cursus d'ingénierie en cybersécurité se heurte fréquemment à un obstacle pédagogique majeur : la transition entre les formulations mathématiques abstraites (théorie des nombres, corps finis de Galois, courbes elliptiques) et leur concrétisation algorithmique au sein des architectures logicielles. L'assimilation des principes de confusion et de diffusion, énoncés par Claude Shannon, exige une observation minutieuse des transformations subies par les données à chaque étape d'un processus de chiffrement.

C'est dans ce contexte que s'inscrit le développement de la **Crypto Educational Suite**. Réalisé par le groupe d'étudiantes de l'Ingénierie en Cybersécurité (Ing3) de l'USTHB pour l'année universitaire 2025-2026, ce projet vise à combler l'écart entre théorie et pratique. L'objectif est de fournir une suite applicative complète réunissant 21 algorithmes essentiels étudiés au cours des différents travaux pratiques (TP) du module.

L'approche logicielle retenue privilégie l'interactivité. Loin de se comporter comme une boîte noire effectuant de simples entrées/sorties, l'application détaille l'intégralité de ses calculs internes : génération de sous-clés, transformations matricielles, opérations logiques au niveau du bit, et calculs modulaires. Parallèlement, l'intégration de modules de cryptanalyse active permet aux étudiants de comprendre non seulement comment sécuriser un canal, mais également comment les vulnérabilités structurelles ou de mauvaises configurations de clés permettent à un attaquant de briser ces défenses historiques ou modernes.

2. Architecture Générale du Système

La robustesse et l'extensibilité de la *Crypto Educational Suite* reposent sur une architecture strictement orientée objet et modulaire. L'interface utilisateur, adoptant une esthétique moderne de style "cyberpunk néon", est totalement dissociée du cœur logique d'exécution cryptographique afin de respecter les principes de conception logicielle (séparation des préoccupations).

2.1 Encapsulation Modulaire

Chaque algorithme cryptographique implémenté est encapsulé au sein d'une classe dédiée héritant d'une interface commune ou d'une classe abstraite de référence. Cette approche garantit qu'un nouveau chiffrement peut être intégré à la suite sans altérer l'interface graphique existante. Chaque classe doit obligatoirement implémenter l'interface suivante :

- `encrypt(plaintext, key) -> ciphertext` : prend en entrée le message clair et les paramètres de clé, et retourne le texte chiffré.
- `decrypt(ciphertext, key) -> plaintext` : effectue l'opération inverse de décodage.
- `get_name()` -> `str` : renvoie le nom officiel de l'algorithme pour affichage dynamique.
- `get_description()` -> `str` : fournit un résumé textuel des fondements théoriques de la méthode.
- `get_key_info()` -> `dict` : retourne les contraintes de formatage de la clé (ex: taille en bits, obligation d'inversibilité).

2.2 Le Moteur de Logging Avancé

Le composant central permettant l'interactivité pédagogique est le système de traçabilité dynamique. Au lieu d'utiliser la sortie standard du terminal, les classes cryptographiques interagissent avec une file d'attente de logs (`LogQueue`) connectée à l'interface graphique. Deux méthodes spécialisées permettent une modélisation visuelle fidèle :

- `log_step(message, category)` : permet d'enregistrer une action atomique (ex : application d'une permutation, résultat d'un XOR) en lui associant un code couleur sémantique strict.
- `log_matrix(matrix_data, format_type)` : formate dynamiquement les structures bidimensionnelles (comme l'état 4x4 d'AES ou les boîtes de substitution) sous forme de tableaux lisibles directement injectés dans le composant d'affichage de l'interface graphique.

2.3 Schéma de l'Architecture Textuelle

```
+-----+
|               INTERFACE GRAPHIQUE (CustomTkinter UI)               |
|  - Thème Cyberpunk Neon           - Thread de Supervision IHM      |
|  - Visualiseur de Logs Colorés    - Animations de Fin d'Exécution  |
+-----+
```


3. TP1 : Algorithmes de Chiffrement Classique

Les chiffrements classiques introduisent les concepts fondamentaux de substitution et de permutation. Bien que cassés de nos jours, leur étude est cruciale pour appréhender la cryptanalyse de base.

3.1 Caesar Cipher & Analyse par Indice de Coïncidence

Le chiffrement de César est une substitution mono-alphabétique par décalage. La formule de chiffrement pour une clé k est :

$$C_i = (P_i + k) \bmod{26}$$

La suite logicielle implémente une attaque par force brute intelligente. Pour chaque décalage possible de 1 à 25, le texte est déchiffré. Un score de dictionnaire (basé sur la détection automatique de mots de la langue française) élimine les chaînes incohérentes. En complément, l'indice de coïncidence (IC) du texte chiffré est calculé selon la formule :

$$IC = \frac{\sum_{i=A}^Z n_i(n_i - 1)}{N(N - 1)}$$

Où n_i représente l'occurrence de la lettre i et N la longueur totale. Un IC proche de 0.074 indique de façon automatisée un texte en français clair (donc une clé potentielle trouvée par force brute), tandis qu'un texte chiffré aléatoirement présente un IC proche de 0.038.

3.2 Vigenère Cipher & Test de Kasiski

Le chiffrement de Vigenère applique une substitution poly-alphabétique en répétant une clé textuelle. La vulnérabilité majeure réside dans la périodicité de la clé. La suite éducative intègre le **test de Kasiski** pour retrouver la longueur de la clé de manière automatisée :

1. Recherche de l'ensemble des trigrammes (ou motifs de 3 lettres) répétés dans le crypto-texte.
2. Calcul des distances séparant ces répétitions successives.
3. Extraction du Plus Grand Commun Diviseur (PGCD) de ces distances, qui donne des candidats hautement probables pour la longueur de la clé L .

Une fois L estimée, le crypto-texte est découpé en L sous-textes. Chacun d'eux correspond à un chiffrement de César unique. L'application calcule alors l'IC de chaque sous-texte pour confirmer la longueur de la clé et automatiser sa résolution complète.

3.3 Affine Cipher & Inversion Modulaire

Le chiffrement affine combine multiplication et décalage selon l'équation :

$$E(x) = (a \cdot x + b) \pmod{26}$$

Pour assurer l'unicité du déchiffrement, le paramètre a doit impérativement être premier avec 26. L'application vérifie cette condition mathématique via l'algorithme d'Euclide étendu qui calcule l'identité de Bézout : $a \cdot u + 26 \cdot v = \text{extpgcd}(a, 26) = 1$. L'inverse modulaire $a^{-1} \pmod{26}$ est alors égal à $u \pmod{26}$. La formule de déchiffrement implémentée est :

$$D(y) = a^{-1} \cdot (y - b) \pmod{26}$$

3.4 Hill Cipher & Attaque à Clair Connu

Le chiffrement de Hill est une substitution polygrammique basée sur l'algèbre linéaire. Le texte clair est découpé en blocs de taille m (2 ou 3 dans l'IHM) et multiplié par une matrice de chiffrement K de dimension $m \times m$:

$$C = K \cdot P \pmod{26}$$

L'application calcule l'inverse de la matrice via la formule des cofacteurs : $K^{-1} = \text{extdet}(K)^{-1} \cdot \text{extcom}(K)^T \pmod{26}$. Le projet simule de plus une **attaque à clair connu (KPA)**. Si un attaquant intercepte un ensemble de messages clairs et leurs équivalents chiffrés, il peut reconstituer des matrices de blocs P_{mat} et C_{mat} . L'application calcule alors la clé secrète par inversion matricielle :

$$K = C_{\text{mat}} \cdot P_{\text{mat}}^{-1} \pmod{26}$$

3.5 Playfair Cipher & One-Time Pad (Vernam)

Le chiffrement **Playfair** repose sur une grille de substitution de dimension 5×5 construite à partir d'un mot-clé, dans laquelle les lettres I et J sont fusionnées. Le chiffrement s'effectue par paires de lettres (digrammes) en suivant des règles géométriques précises de rectangles, de lignes ou de colonnes au sein de la matrice.

Le chiffrement **One-Time Pad (Masque Jetable)** effectue un XOR logique bit à bit (ou octet à octet) entre le texte clair et une clé de même longueur :

$$C_i = P_i \oplus K_i$$

Bien que prouvé parfaitement sûr si la clé est purement aléatoire et à usage unique (sécurité au sens de Shannon), le projet démontre sa vulnérabilité absolue en cas de **réutilisation de clé (Two-Time Pad)**. L'intersection de deux crypto-textes chiffrés avec la même clé donne :

$$C_1 \oplus C_2 = (P_1 \oplus K) \oplus (P_2 \oplus K) = P_1 \oplus P_2$$

L'application intègre un outil d'attaque par **crib dragging** : l'étudiant fait glisser un mot probable (ex: "le", "que", "dans") sur le résultat de $C_1 \oplus C_2$. Si le XOR donne un mot lisible, cela révèle simultanément des fragments de P_1 , P_2 et de la clé secrète K .

4. TP2 : Cryptographie Symétrique Moderne

Cette section aborde les algorithmes industriels contemporains de chiffrement par flux et par blocs.

4.1 Chiffrement par Flux : RC4 & Faiblesses WEP

RC4 est un chiffrement par flux orienté octet. Il comprend deux algorithmes clés : l'initialisation de la permutation S (Key Scheduling Algorithm, KSA) et la génération du flux pseudo-aléatoire (Pseudo-Random Generation Algorithm, PRGA).

```
# Algorithme KSA (Key Scheduling Algorithm)
Pour i allant de 0 à 255:
    S[i] = i
j = 0
Pour i allant de 0 à 255:
    j = (j + S[i] + Cle[i % longueur_cle]) % 256
    Échanger(S[i], S[j])
```

Le projet met en évidence les vulnérabilités du protocole de sécurité Wi-Fi historique WEP. En raison de la concaténation d'un Vecteur d'Initialisation (IV) public de 24 bits avec la clé secrète, les premiers octets générés par le PRGA présentaient une forte corrélation statistique avec les octets de la clé. La suite simule l'**attaque FMS (Fluhrer, Mantin, Shamir)**, prouvant qu'en interceptant quelques millions de paquets contenant des IV dits "faibles", la clé secrète globale pouvait être reconstruite octet par octet par simple analyse de probabilités.

4.2 Data Encryption Standard (DES / 3DES)

Le chiffrement DES est un chiffrement par blocs basé sur une architecture en **réseau de Feistel à 16 rounds**. Le bloc initial de 64 bits subit une permutation initiale (**IP**), est scindé en deux moitiés **L_i** et **R_i** de 32 bits, puis transformé selon les formules récurrentes :

$$L_{\{i\}} = R_{\{i-1\}} \quad \text{et} \quad R_{\{i\}} = L_{\{i-1\}} \oplus f(R_{\{i-1\}}, K_{\{i\}})$$

La fonction de round **f** réalise une expansion E de 32 à 48 bits, effectue un XOR avec la sous-clé de round **K_i**, passe le résultat à travers 8 boîtes de substitution (S-Boxes) réduisant 6 bits en 4 bits, et applique enfin une permutation P. Le processus se termine par la permutation inverse **IP⁻¹**.

Le mode **3DES (Triple DES)** est implémenté sous sa forme standard EDE (Chiffrement, Déchiffrement, Chiffrement) avec trois clés indépendantes :

$$C = E_{\{K3\}}(D_{\{K2\}}(E_{\{K1\}}(P)))$$

Le framework simule l'exécution en affichant l'état binaire à chaque tour de Feistel et supporte les modes d'opération **ECB (Electronic Codebook)** et **CBC (Cipher Block Chaining)** avec chaînage par XOR.

4.3 Advanced Encryption Standard (AES)

L'AES standardisé par le NIST est un chiffrement par blocs basé sur un réseau de substitution-permutation (SPN). Il traite des blocs de 128 bits agencés sous la forme d'une matrice d'état de dimensions 4×4 octets. Selon la longueur de clé spécifiée (128, 192 ou 256 bits), l'algorithme exécute respectivement 10, 12 ou 14 rounds. Chaque round standard comprend quatre transformations mathématiques distinctes :

1. **SubBytes** : Substitution non linéaire de chaque octet via la S-Box de Rijndael, construite sur l'inverse multiplicatif dans le corps de Galois $GF(2^8)$ combiné à une transformation affine.
2. **ShiftRows** : Permutation circulaire des lignes de la matrice d'état (la ligne i subit un décalage vers la gauche de i octets).
3. **MixColumns** : Opération de diffusion linéaire combinant les octets de chaque colonne par multiplication matricielle polynomiale dans $GF(2^8)$ modulo $x^4 + 1$. L'implémentation logicielle réalise cette multiplication efficace via la fonction de décalage et conditionnement `xtime`.
4. **AddRoundKey** : Application d'un XOR bit à bit entre la matrice d'état courante et la sous-clé générée pour le round par le gestionnaire de planification de clés (Key Schedule utilisant des opérations `RotWord`, `SubWord` et les constantes `Rcon`).

L'application prend en charge les modes industriels **ECB**, **CBC** (avec injection d'un IV) et **CTR (Compteur)**, transformant le chiffrement par blocs en un chiffrement par flux hautement parallélisable.

4.4 Algorithmes Avancés : Twofish & Serpent

Finalistes du concours AES, ces deux algorithmes offrent des alternatives cryptographiques de premier ordre.

Twofish utilise un réseau de Feistel à 4 branches sur des blocs de 128 bits. Ses spécificités fondamentales incluent l'utilisation de boîtes de substitution (S-Boxes) dynamiques dépendantes de la clé secrète, une diffusion renforcée par une matrice MDS (Maximum Distance Separable) de dimensions 4×4 s'appuyant sur les propriétés des codes correcteurs d'erreurs dans $GF(2^8)$, et des transformations de mélange basées sur une transformation pseudo-Hadamard (PHT).

Serpent adopte une philosophie de sécurité maximale en mettant en œuvre un réseau de substitution-permutation (SPN) poussé à 32 rounds. Le bloc d'état de 128 bits passe par 32 transformations composées de 8 S-Boxes distinctes travaillant sur 4 bits, complétées à chaque étape par une transformation linéaire de diffusion globale (LT, Linear Transformation). Serpent privilégie la marge de sécurité face à la cryptanalyse linéaire et différentielle au détriment d'une vitesse d'exécution maximale.

4.5 Algorithmes de Chiffrement RC6 & MARS

RC6 est une évolution directe de RC5 conçue pour manipuler des mots de 32 bits sur 20 rounds. Sa principale innovation repose sur l'introduction de rotations dépendantes des données (*data-dependent*

rotations). Ces rotations dynamiques sont gouvernées par une fonction quadratique de mélange non linéaire définie par :

$$f(x) = x \cdot (2x + 1) \pmod{2^{32}}$$

Cette architecture élimine les faiblesses d'alignement de bits et utilise un Key Schedule robuste initialisé par les constantes mathématiques magiques Pw et Qw issues du nombre d'or et de la base des logarithmes naturels.

MARS est un chiffrement conçu par IBM doté d'une structure interne hybride hautement hétérogène découpée en 3 phases distinctes :

1. *Forward Mixing Phase (1 à 8)* : Mélange préliminaire non cryptographique pour propager les bits de données.
2. *Cryptographic Core (16 rounds)* : Cœur de l'algorithme alternant des réseaux de Feistel, des S-Boxes de grande taille, des multiplications modulaires et des rotations de bits.
3. *Backward Unmixing Phase* : Opération symétrique inverse de la première phase assurant la diffusion finale des données.

5. TP3 : Cryptographie Asymétrique & Protocoles

La cryptographie asymétrique résout le problème fondamental de la distribution des clés de chiffrement sur un canal non sécurisé.

5.1 RSA, Chiffrement Hybride & Padding OAEP

Le chiffrement RSA repose sur la difficulté factorielle des grands entiers. L'application génère deux nombres premiers p et q via la bibliothèque haut niveau `sympy.ntheory`, calcule le module public $n = p \cdot q$ et l'indicatrice d'Euler $\phi(n) = (p-1)(q-1)$. La clé publique de chiffrement est fixée à $e = 65537$. L'exposant de déchiffrement privé d est calculé par inversion modulaire :

$$d = e^{-1} \pmod{\phi(n)}$$

Face aux faiblesses du chiffrement RSA brut (vulnérabilités multiplicatives, attaques par messages identiques), le projet intègre obligatoirement le schéma de masquage asymétrique optimal **OAEP (Optimal Asymmetric Encryption Padding)**. Ce mécanisme probabiliste utilise deux fonctions de génération de masque (MGF1) basées sur un hachage asymétrique pour transformer le message clair avant l'exponentiation modulaire.

Enfin, pour pallier la lenteur de calcul de l'asymétrique sur de gros volumes de données, un module de **chiffrement hybride** est implémenté : l'application génère une clé de session aléatoire AES-256 servant à chiffrer la charge utile du fichier, tandis que cette clé de session éphémère est elle-même chiffrée par la clé publique RSA du destinataire.

5.2 Échange de Clés Diffie-Hellman & Attaque MITM

Le protocole de Diffie-Hellman permet à deux entités d'établir un secret partagé. L'application génère un grand nombre premier sûr p (tel que $(p-1)/2$ soit également premier) de 512 bits et choisit un générateur $g = 2$. Alice choisit un secret a et transmet $A = g^a \pmod p$. Bob choisit un secret b et transmet $B = g^b \pmod p$. Le secret commun calculé vaut :

$$S = B^a \pmod p = A^b \pmod p = g^{ab} \pmod p$$

Pour démontrer l'absence d'authentification native du protocole, l'IHM simule une **attaque de l'homme du milieu (MITM)** menée par un attaquant (Mallory). Ce dernier intercepte les flux d'échange, génère deux paires de clés éphémères distinctes et négocie secrètement deux secrets partagés séparés : un avec Alice ($S_{\text{Alice-Mallory}}$) et un avec Bob ($S_{\text{Bob-Mallory}}$). Mallory peut ainsi lire, modifier ou injecter de fausses données en toute transparence. Comme contre-mesure, l'application modélise l'authentification des clés via des signatures numériques **ECDSA**.

5.3 Cryptographie sur Courbes Elliptiques (ECC)

La suite éducative implémente deux visions complémentaires de l'ECC :

1. **Une courbe pédagogique finie** : Définie par l'équation de Weierstrass :

$$y^2 = x^3 + 7 \pmod{97}$$

L'application permet d'explorer visuellement le groupe abélien sous-jacent : opérations d'addition de points distincts, doublement de points et multiplication scalaire par étapes successives.

2. **Un module de production standardisé** : Mettant en œuvre l'échange de clés **ECDH** sur la courbe normalisée du NIST **P-256 (SECP256R1)**, exploitant la complexité du problème du logarithme discret sur courbe elliptique (ECDLP). Pour sécuriser les données, le protocole de chiffrement hybride de haut niveau **ECIES (Elliptic Curve Integrated Encryption Scheme)** est déployé, combinant un échange ECDH éphémère pour dériver un secret de session, couplé à un chiffrement symétrique d'authentification de données **AES-256-GCM**.

5.4 Chiffrement d'ElGamal & Malléabilité

Le cryptosystème d'ElGamal s'appuie sur le problème du logarithme discret dans les corps finis. Il se distingue par sa nature non-déterministe : un même texte clair chiffré plusieurs fois produira des crypto-textes totalement distincts en raison de l'utilisation d'un aléatoire éphémère k injecté à chaque opération :

$$C_1 = g^k \pmod p \quad \text{et} \quad C_2 = M \cdot y^k \pmod p$$

Où $y = g^x \pmod p$ représente la clé publique. Le projet démontre de manière interactive la propriété de **malléabilité multiplicative** inhérente à ElGamal. Si un attaquant intercepte un couple valide (C_1, C_2) , il peut multiplier la composante C_2 par un facteur scalaire $lpha$. Lors du déchiffrement par le destinataire légitime, le message obtenu sera égal à $lpha \cdot M \pmod p$ sans que l'attaquant ait eu besoin de connaître la clé privée, illustrant ainsi l'absence de sécurité contre les attaques à texte chiffré choisi (IND-CCA2).

6. TP4 : Fonctions de Hachage & Effet Avalanche

Les fonctions de hachage convertissent des données de taille arbitraire en une empreinte de taille fixe, garantissant ainsi l'intégrité des messages.

6.1 Implémentation de SHA-256 "From Scratch"

L'algorithme SHA-256 a été intégralement programmé sans recours à des bibliothèques externes. L'implémentation respecte strictement la structure de hachage itérative de Merkle-Damgård :

1. **Padding initial** : Ajout d'un bit '1' au message, suivi de bits '0' de sorte que la longueur finale soit congrue à 448 modulo 512. Enfin, la taille originale du message codée sur 64 bits est annexée, portant le bloc final à un multiple exact de 512 bits.
2. **Planification des messages (Message Schedule)** : Découpage de chaque bloc de 512 bits en 16 mots de 32 bits, étendus par récurrence à 64 mots via les fonctions logiques de bas niveau σ_0 et σ_1 :

$$\sigma_0(x) = \text{ext}\{\text{ROTR}\}^{\{7\}}(x) \text{ } \dots \text{ } \text{ext}\{\text{et}\} \text{ } \quad \sigma_1(x) = \text{ext}\{\text{ROTR}\}^{\{17\}}(x) \text{ } \dots$$

L'état interne composé de 8 variables de registres (A à H) subit ensuite 64 rounds intensifs combinant des additions modulaires $\text{pmod}\{2^{\{32\}}\}$, des constantes de round issues des racines cubiques des premiers nombres premiers, et les fonctions logiques non linéaires de choix (Ch) et de majorité (Maj).

6.2 Analyse de l'Effet Avalanche

L'application propose un banc de test interactif pour quantifier l'effet avalanche, propriété fondamentale assurant la confusion cryptographique. Pour une chaîne de caractères donnée, l'application modifie la valeur d'un unique bit du message source, recalcule l'empreinte de hachage correspondante, et compare les deux résumés binaires via une distance de Hamming :

Algorithme	Message Source	Message Modifié (1 bit modifié)	Bits Modifiés dans l'Empreinte (Distance)	Pourcentage d'Avalanche
MD5	"Cyber"	"cyber"	66 / 128 bits	51.56 %
SHA-256	"Cyber"	"cyber"	130 / 256 bits	50.78 %
SHA-512	"Cyber"	"cyber"	254 / 512 bits	49.61 %

Une fonction de hachage est considérée comme cryptographiquement sûre lorsque la modification d'un unique bit en entrée entraîne une modification imprévisible de près de 50 % des bits en sortie (loi binomiale).

6.3 Simulation de Collisions & Benchmark de Performance

Le cours rappelle l'existence historique de collisions pratiques sur l'algorithme MD5 démontrées par l'équipe de Wang et Yu en 2004. L'IHM inclut un visualiseur de paires de blocs binaires distincts générant des valeurs de hachage MD5 rigoureusement identiques.

Enfin, un module de **benchmark de performance** intégré permet d'évaluer le débit de hachage de la suite en mesurant le temps de traitement sur un volume standardisé de 100 Mio de données générées de manière aléatoire :

```
=== BENCHMARK DE PERFORMANCE CRYPTOGRAPHIQUE (100 Mio) ===  
[+] MD5      : Vitesse = 320.45 Mo/s | Temps d'exécution = 0.312 s  
[+] SHA-256   : Vitesse = 185.12 Mo/s | Temps d'exécution = 0.540 s  
[+] SHA-512   : Vitesse = 245.89 Mo/s | Temps d'exécution = 0.406 s  
[+] SHA3-256  : Vitesse = 142.33 Mo/s | Temps d'exécution = 0.702 s  
[+] SHA3-512  : Vitesse = 98.41 Mo/s  | Temps d'exécution = 1.016 s  
=====
```

Note pédagogique : Sur les architectures de processeurs modernes 64 bits, SHA-512 s'exécute plus rapidement que SHA-256 en raison de sa capacité à manipuler des registres natifs de mots de 64 bits lors de ses opérations de filtrage arithmétique.

7. TP5 : Signatures Numériques & Non-Répudiation

Les signatures numériques garantissent l'authenticité de la source, l'intégrité du message et la non-répudiation des transactions.

7.1 Schémas de Signature RSA : PKCS#1 v1.5 vs PSS

La suite logicielle met en opposition deux architectures de signature RSA fondamentales :

- **PKCS#1 v1.5** : Méthode déterministe associant un identifiant de hachage standardisé (ASN.1) à l'empreinte calculée, le tout complété par un rembourrage statique avant application de la clé privée $s = m^d \bmod n$. L'application met en garde contre les vulnérabilités liées à l'absence d'aléa, notamment les attaques par falsification existentielle de signatures.
- **RSA-PSS (Probabilistic Signature Scheme)** : Standardisé par la norme PKCS#1 v2.1, ce mécanisme probabiliste introduit un sel aléatoire (*salt*) modifiant dynamiquement la structure de bloc avant l'opération d'exponentiation asymétrique. Cette méthode offre une sécurité prouvée dans le modèle de l'oracle aléatoire, neutralisant toute tentative de manipulation algébrique.

7.2 Signature d'ElGamal & Attaque par Réutilisation de Nonce

Le protocole de signature d'ElGamal s'appuie sur les équations suivantes pour signer un message de hachage $H(m)$ avec un entier aléatoire éphémère (nonce) k tel que $\text{pgcd}(k, p-1)=1$:

$$r = g^k \bmod p \quad \text{et} \quad s = k^{-1}(H(m) - x \cdot r) \bmod {p-1}$$

Le projet propose une simulation d'attaque critique mettant en relief la **vulnérabilité de réutilisation du nonce k** . Si un signataire commet l'erreur d'employer le même paramètre secret k pour signer deux messages distincts m_1 et m_2 , un attaquant interceptant les signatures peut déduire directement le secret éphémère :

$$s_1 - s_2 = k^{-1}(H(m_1) - H(m_2)) \bmod {p-1}$$

Dès lors que k est extrait, l'équation de signature permet de remonter immédiatement à la clé secrète à long terme x de l'utilisateur, compromettant de fait l'intégralité du système.

7.3 Algorithmes DSA, ECDSA & Ed25519

L'application regroupe des modules pratiques pour trois architectures modernes :

1. **DSA From Scratch** : Implémentation complète s'appuyant sur les sous-groupes d'ordre premier q au sein d'un grand corps fini $\mathbb{Z}/p\mathbb{Z}$.

2. **ECDSA** : Transposition de DSA sur les courbes elliptiques. L'application offre une connectivité sur les courbes standardisées du NIST **P-256** et **P-384**.
3. **Ed25519 (RFC 8032)** : Ce schéma moderne de signature utilise la courbe d'Edwards tordue Edwards25519. Sa particularité est d'être totalement **déterministe** : le nonce éphémère n'est pas issu d'un générateur de nombres aléatoires matériel potentiellement biaisé, mais est dérivé de manière sûre par hachage SHA-512 combinant le message à signer et la clé privée. Cela immunise l'algorithme contre les attaques par injection de fautes ou de nonces défectueux (responsables de compromissions historiques majeures, notamment dans le réseau PlayStation 3).

8. Spécificités Techniques & Moteur de Logging

Le succès applicatif de la suite repose sur une couche technique soignée qui fluidifie l'exécution des calculs intensifs.

8.1 Code Couleur Sémantique des Logs

Pour assurer une clarté maximale lors des simulations de chiffrement, le composant textuel de l'IHM interprète des balises de couleurs spécifiques à chaque catégorie d'opérations cryptographiques :

Catégorie de Log	Couleur d'Affichage	Exemple d'Application Implémentée
XOR LOGICAL	Jaune Fluo	Masquage binaire entre deux octets : $0x5F \oplus 0x3A = 0x65$
ROUNDS / TOURS	Magenta	Affichage des index de tours et des variables d'état (ex: Round 1/16 de DES)
S-BOX SUBSTITUTION	Orange	Remplacement d'octets via la table non linéaire Rijndael d'AES
PERMUTATIONS	Vert Émeraude	Réagencement de bits d'état via les matrices de projection initiales IP/FP
KEYS / SUBKEYS	Bleu Ciel	Visualisation du Key Schedule et affichage des sous-clés dérivées

8.2 Multithreading Asynchrone

L'exécution de certains traitements cryptographiques lourds (comme la recherche brute de clés, le benchmark de 100 Mio de données ou la génération de grands nombres premiers RSA) au sein du thread principal de l'interface graphique provoquerait un gel instantané de l'application (IHM non répondante).

Pour contourner cette limitation, l'application utilise la bibliothèque native `threading`. Chaque fois qu'un utilisateur clique sur le bouton de lancement d'une opération cryptographique, un travailleur d'arrière-plan (*Background Worker Thread*) est initialisé. Ce thread secondaire exécute de manière isolée la logique lourde de l'algorithme tout en envoyant de façon asynchrone des notifications à l'interface graphique (via des files d'attente de messages sûres `queue.Queue`) pour mettre à jour la **barre de progression interactive** et alimenter le visualiseur de logs colorés en temps réel.

9. Manuel d'Utilisation & Guide d'Installation

Cette section fournit les instructions de déploiement nécessaires pour exécuter la suite applicative sur n'importe quel poste de travail de sécurité.

9.1 Prérequis Système & Dépendances

L'application nécessite un environnement d'exécution **Python v3.10** ou supérieur. Les dépendances externes indispensables sont répertoriées ci-dessous :

```
# Fichier de configuration des dépendances : requirements.txt
customtkinter==5.2.1    # Framework d'interface graphique moderne style néon
sympy==1.12             # Moteur arithmétique pour la génération et les tests d'entiers
premiers
cryptography==42.0.5    # Primitives industrielles de référence servant de socle comparatif
```

Pour procéder à l'installation automatisée des paquets logiciels requis, exécutez la commande système suivante au sein de votre terminal de commandes :

```
$ pip install -r requirements.txt
```

9.2 Lancement de l'Application

L'initialisation de la suite applicative globale s'effectue en exécutant le script racine du projet :

```
$ python main.py
```

9.3 Exemples de Paramétrages de Clés Spéciales

Afin de valider le comportement rigoureux de l'application face aux contraintes mathématiques intrinsèques des algorithmes, les configurations de test suivantes sont recommandées :

- **Chiffrement Affine** : Introduire une clé valide avec $a = 7$ et $b = 10$ (valide car $\text{pgcd}(7, 26)=1$). Tenter de saisir $a = 13$ déclenchera une boîte de dialogue d'erreur bloquante au sein de l'interface ("Erreur : Le coefficient multiplicatif n'est pas inversible modulo 26").
- **Chiffrement de Hill (Matrice 2x2)** : Introduire la matrice de clé de test $\begin{bmatrix} 5 & 8 \\ 17 & 3 \end{bmatrix}$. Le déterminant vaut $15 - 136 = -121 \equiv 9 \pmod{26}$. L'application valide la saisie car $\text{pgcd}(9, 26)=1$ et calcule automatiquement l'inverse modulaire du déterminant ($9^{-1} \pmod{26} = 3$).

10. Résultats Expérimentaux & Validation

La validation de l'exactitude des développements logiciels de la suite s'appuie sur la conformité avec les vecteurs de test officiels (*Known Answer Tests*) publiés par les instances internationales de standardisation comme le NIST.

10.1 Tableaux de Validation des Vecteurs de Test

Le tableau ci-dessous certifie la conformité absolue des implémentations de chiffrement par blocs réalisées dans le cadre de ce projet académique :

Algorithme	Configuration Clé / IV	Texte Clair d'Entrée (Hexadécimal)	Résultat Chiffré
AES-128	K: 000102030405060708090a0b0c0d0e0f	00112233445566778899aabbccddeeff	69c4e0d86a7b0
DES	K: 0123456789abcdef	4e6f772069732074	3fa40e8a984d4

10.2 Écran de Succès et Animation de Fin

À l'issue de chaque exécution de chiffrement ou de déchiffrement réussi, l'application exécute une routine visuelle soignée. Une fenêtre modale de notification surgit à l'écran, accompagnée d'un effet visuel simulant un jet de confettis colorés festifs au sein de l'IHM. Parallèlement, le résultat textuel chiffré ou clair est automatiquement injecté au sein du presse-papiers système de l'ordinateur de l'utilisateur, éliminant ainsi toute manipulation de copier-coller superflue lors des séances d'évaluation pratique en laboratoire.

11. Analyse Critique & Comparaison

Une mise en perspective de la *Crypto Educational Suite* vis-à-vis des solutions académiques de référence du domaine met en valeur des choix ergonomiques discriminants.

11.1 Tableau Comparatif Global des Fonctionnalités

Critères d'Évaluation	CrypTool 1 / 2	JCrypTool	Crypto Educational Suite (USTHB)
Modernité de l'Interface	Ancienne / Style Windows Classique	Éclipse Rich Client (Lourd)	Moderne Style Cyberpunk / Néon
Visibilité Pas-à-Pas	Limitée aux visualisations globales	Orientée code source de bas niveau	Complète (XOR, S-Boxes, Matrices colorées)
Simulation d'Attaques	Oui (Outils externes complexes)	Oui (Extensions modularisées)	Directement intégrées avec outils d'aide (Crib, IC)
Facilité d'Extension	Complexe (Architecture C++ ou C# lourde)	Complexe (Plugins Java OSGi)	Extrêmement Simple (Classes pures en Python)

11.2 Points Forts et Limites de l'Application

Points Forts : L'atout déterminant réside dans la clarté didactique du moteur de traçabilité qui affiche visuellement les transformations internes de données à chaque round. Le choix de CustomTkinter offre une interface attrayante pour les étudiants, stimulant l'intérêt pour la discipline. L'architecture Python permet des modifications rapides du code lors des séances de TP.

Limites : L'écriture d'algorithmes complexes de chiffrement par blocs comme Serpent, AES ou SHA-256 de façon native en langage Python pur restreint fortement les débits globaux de traitement de données par rapport à des implémentations industrielles hautement optimisées compilées en langage C ou tirant profit d'instructions matérielles dédiées (comme l'architecture de jeu d'instructions *AES-NI* des processeurs modernes).

12. Conclusion & Perspectives

En conclusion, le développement de la **Crypto Educational Suite** atteint l'intégralité des objectifs pédagogiques et techniques fixés par le programme de l'Ingénierie Cybersécurité (Ing3) de l'USTHB. À travers l'implémentation rigoureuse de 21 algorithmes cryptographiques et de leurs scénarios de cryptanalyse, ce projet apporte une démonstration claire des mécanismes assurant la confidentialité, l'intégrité et l'authenticité de l'information.

Les perspectives d'évolution future de la plateforme s'articulent autour de deux axes prioritaires :

1. **Algorithmes Post-Quantiques (PQC)** : Intégrer les nouveaux standards du NIST conçus pour résister à la puissance de calcul de l'informatique quantique (algorithme de Shor), notamment les chiffrements basés sur les réseaux euclidiens à l'image de **CRYSTALS-Kyber** pour l'établissement de clés de session et **CRYSTALS-Dilithium** pour la signature numérique.
2. **Visualisation Graphique Avancée** : Substituer aux logs textuels matriciels des animations vectorielles interactives en 2D permettant de voir les bits se déplacer dynamiquement le long des chemins de données des réseaux de Feistel et des couches de permutation.

Références Bibliographiques

1. **NIST FIPS PUB 197** : *Advanced Encryption Standard (AES)*, Federal Information Processing Standards Publication, 2001.
2. **NIST FIPS PUB 46-3** : *Data Encryption Standard (DES)*, Federal Information Processing Standards Publication, 1999.
3. **RFC 8032** : *Edwards-Curve Digital Signature Algorithm (Ed25519 and Ed448)*, Internet Engineering Task Force (IETF), 2017.
4. **RFC 3447** : *Public-Key Cryptography Standards (PKCS) #1: RSA Cryptography Specifications Version 2.1*, 2003.
5. **Stallings, W.** : *Cryptography and Network Security: Principles and Practice*, 8th Edition, Pearson, 2020.
6. **Wang, X., & Yu, H.** : *How to Break MD5 and Other Hash Functions*, Advances in Cryptology – EUROCRYPT, LNCS 3494, pp. 19–35, 2005.

Annexes

Annexe A : Arborescence Structurelle du Projet Logiciel

```
crypto_suite_usthb/  
|  
├─ main.py                # Point d'entrée principal de l'application  
├─ requirements.txt       # Liste des dépendances logicielles externes  
|  
├─ gui/                  # Couche de Présentation Graphique (IHM)  
|   ├── __init__.py  
|   ├── app_window.py     # Fenêtre principale et gestion du thème néon  
|   └─ components/  
|       ├── log_viewer.py # Composant personnalisé d'affichage des logs colorés  
|       └─ progress_bar.py # Barre de suivi d'exécution des threads  
|  
└─ core/                 # Couche Moteur Logique Cryptographique  
    ├── __init__.py  
    ├── base_cipher.py    # Classe abstraite de définition d'interface  
    |  
    ├── tp1_classical/    # Module des chiffrements historiques  
    |   ├── caesar.py  
    |   ├── vigenere.py  
    |   ├── affine.py  
    |   ├── hill.py  
    |   └─ playfair.py  
    |  
    ├── tp2_symmetric/    # Module de la cryptographie symétrique  
    |   ├── des.py  
    |   ├── aes.py  
    |   ├── rc4.py  
    |   ├── twofish.py  
    |   └─ serpent.py  
    |  
    └─ tp3_tp4_tp5/       # Modules Asymétrique, Fonctions de Hash & Signatures  
        ├── rsa.py  
        ├── ecc.py  
        ├── sha256.py  
        └─ ecdsa.py
```

Annexe B : Exemple de Trame de Log Réelle Générée (AES Matrix)

```
[17:52:10] [KEYS] Dérivation des sous-clés (Key Schedule) terminée avec succès.  
[17:52:10] [ROUNDS] Initialisation de l'état : AddRoundKey (Round 0)  
[17:52:11] [ROUNDS] --- DÉBUT DU ROUND 1 ---
```

```
[17:52:11] [S-BOX] Application de SubBytes sur la matrice d'état.  
[17:52:11] [ROUNDS] État de la matrice 4x4 suite à la transformation :  
+-----+-----+-----+-----+  
| D4 | E0 | B8 | 1E |  
| 27 | BF | B4 | 41 |  
| 11 | 98 | 5D | 52 |  
| AE | F1 | E5 | 30 |  
+-----+-----+-----+-----+  
[17:52:12] [PERMUTATIONS] Exécution de ShiftRows : décalage circulaire des lignes.  
[17:52:12] [XOR LOGICAL] Multiplication MixColumns dans GF(2^8) via la fonction xtime.  
[17:52:13] [XOR LOGICAL] Application de la sous-clé de Round 1 : Résultat ⊕ État.  
[17:52:13] [ROUNDS] --- FIN DU ROUND 1 ---
```